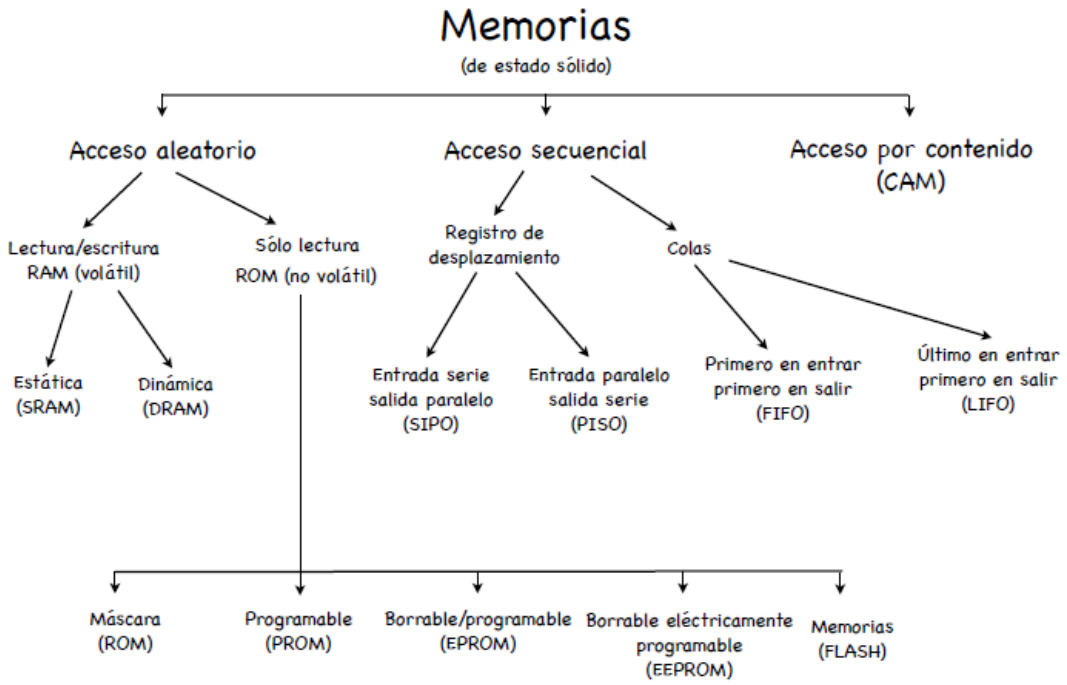
# T10. Memorias de estado sólido

## Memorias




Las memorias (de estado sólido) se pueden clasificar según,

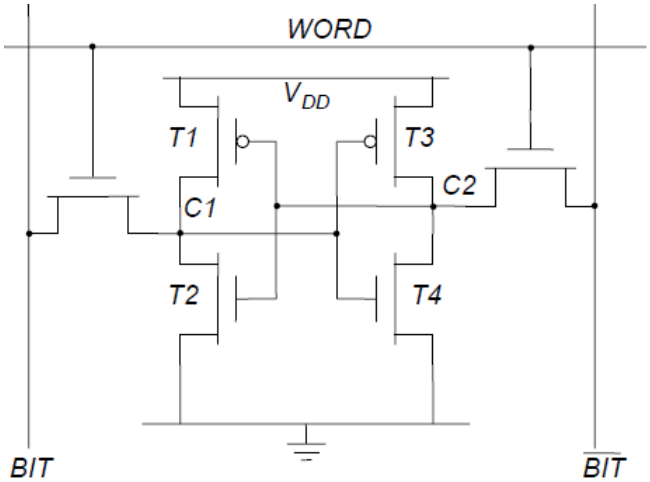
1. El modo de acceso:
    a. **Acceso aleatorio** (RAM/ROM): se puede acceder a cualquier posición directamente.
    b. **Acceso secuencial**: se accede en orden, uno tras otro (registros de desplazamiento, FIFO, LIFO).
    c. **Acceso por contenido** (CAM): se accede por el **valor** almacenado, no por la dirección.
2. El tipo de operación:
    a. **Lectura y escritura (RAM – volátil).**
        i. **SRAM (Static RAM)**: No necesita refresco, es rápida y cara. Se usa en memorias caché.
        ii. **DRAM (Dynamic RAM)**: Necesita refresco periódico, es más lenta, más densa y barata. Se usa como memoria principal.
    b. **Sólo lectura (ROM – no volátil).**
        i. **ROM (Read Only Memory)**: el contenido se graba en fábrica y no se puede modificar.
        ii. **PROM (Programmable ROM)**: programable una sola vez por el usuario.
        iii. **EPROM (Erasable PROM)**: borrable con luz ultravioleta y reprogramable.
        iv. **EEPROM (Electrically Erasable PROM)**: borrado eléctrico, permite varias reescrituras.
        v. **FLASH**: evolución de EEPROM, más rápida y usada en USB, tarjetas SD, SSDs, etc.

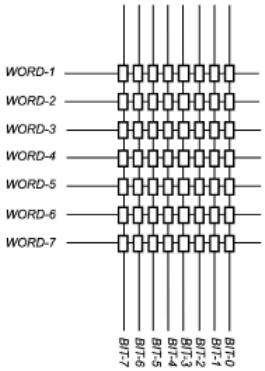
3. La organización:
    a. **Registros de desplazamiento**:
        i. **SIPO (Serial In, Parallel Out)**: los bits entran uno a uno y salen todos a la vez.
        ii. **PISO (Parallel In, Serial Out)**: los bits entran todos juntos y salen uno a uno.
        iii. **FIFO (First In, First Out)**: Cola, el primer dato que entra es el primero que sale.
        iv. **LIFO (Last In, First Out)**: Pila, el último dato que entra es el primero que sale.

## Celda RAM estática (SRAM)

Una celda de memoria **SRAM** (RAM estática) está diseñada para almacenar un solo **bit de información**, es decir, un $0$ o un $1$. Lo que la diferencia de otros tipos de memorias es que no necesita refrescarse periódicamente (como la DRAM) y permite leer y escribir datos de manera muy rápida. Cada celda SRAM está formada por **seis transistores**, aunque lo esencial para entender su funcionamiento son dos partes clave:

1. Un **bistable** (una pareja de inversores cruzados) que mantiene el valor almacenado mientras haya alimentación.
2. Dos **transistores de acceso** que permiten conectar la celda al exterior (a las líneas de datos $BIT$ y $\overline{BIT}$) solo cuando se activa la línea de selección WORD.




<u>Modo escritura (guardar un valor)</u>

Para **escribir** un dato en la celda, el circuito realiza dos acciones:

1. El valor que se quiere almacenar se pone en las líneas de datos. Por ejemplo, si queremos guardar un $1$, se pone un $1$ lógico en la línea $BIT$ y un $0$ en $\overline{BIT}$. Estas dos líneas son complementarias.

2. Se activa la **señal WORD**, que selecciona esa celda concreta y habilita los transistores de acceso. Entonces, las líneas $BIT$ y $\overline{BIT}$ **sobrescriben** el

contenido de la celda: el valor se fuerza a través de los transistores y se impone sobre el bistable, haciendo que se quede almacenado.

Este proceso es **destructivo** (se sobrescribe) pero estable: una vez escrita, la celda mantiene su estado hasta que se vuelva a escribir algo diferente.

<u>Modo lectura (consultar el valor)</u>

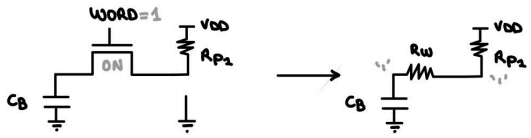
Antes de leer, el circuito **precarga** ambas líneas $BIT$ y $\overline{BIT}$ a un valor **alto** (por ejemplo, $1$ lógico). A continuación, se activa de nuevo la señal WORD, lo que conecta la celda a esas líneas.
Ahora ocurre lo siguiente: según el valor almacenado en la celda (es decir, el estado interno de los nodos C1 y C2), uno de ellos está a nivel bajo ($0$) y el otro a nivel alto ($1$). Entonces, a través del transistor de acceso correspondiente, uno de los **nodos** empezará a **descargar la línea** que estaba **precargada a** $1$.

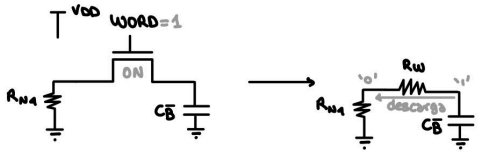
Por ejemplo, si C2 está a $0$, entonces $\overline{BIT}$ (que inicialmente estaba a $1$) comenzará a descargarse a través del transistor. Este proceso es controlado y rápido, y al detectar qué línea baja su voltaje, el circuito de lectura puede saber si el valor almacenado era un $0$ o un $1$.
Este proceso no destruye el contenido de la celda: el biestable no cambia, simplemente se detecta su valor.

<u>Comportamiento eléctrico: carga y descarga</u>

Este proceso puede entenderse mejor si pensamos en términos eléctricos: los nodos internos de la celda (como C2) se comportan como **condensadores** que pueden cargarse o descargarse a través de resistencias cuando se conectan a las líneas $BIT$ o $\overline{BIT}$.
En lectura, cuando activamos WORD, uno de los nodos (por ejemplo, C2) se conecta a $\overline{BIT}$. Si C2 está a $0$ y $\overline{BIT}$ se había precargado a $1$, el condensador absorbe carga, provocando una caída de tensión en $\overline{BIT}$. Esa caída de tensión es lo que detecta el circuito de lectura. Si modelamos esto como una resistencia en serie con el condensador, veremos que hay dos resistencias:

- Una $R_W$ que representa el camino desde la línea $BIT$ hacia el nodo.
- Una $R_N$ que representa la resistencia en el nodo hacia tierra.



Si $R_N$ es **mucho menor** que $R_W$ (es decir, si el nodo está bien conectado a tierra), entonces la mayoría de la caída de tensión ocurre en $R_W$, lo que implica que el voltaje en $BIT$ cae bruscamente. Así es como se consigue que se detecte claramente un $0$. En cambio, si $R_W$ es pequeña y $R_N$ es grande, la caída se reparte o incluso apenas se nota en $BIT$, y el circuito podría no detectar correctamente el valor.
Por tanto, para que se detecte bien un $0$, es fundamental que la caída de tensión ocurra principalmente en la línea $R_W$, y eso se consigue si se cumple que $R_W$ es mucho mayor que $R_N$.

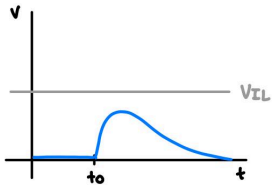
*Lógica de niveles: el umbral* $V_{IL}$

En los sistemas digitales, el receptor interpreta una señal como un $0$ si la tensión baja de cierto valor llamado $V_{IL}$ (Voltage Input Low). En la lectura de la celda, si la caída de tensión en la línea $BIT$ o $\overline{BIT}$ baja por debajo de $V_{IL}$, el circuito sabe que se está leyendo un $0$.

Evolución de la tensión en el tiempo

Cuando se activa la lectura o la escritura (es decir, cuando se activa WORD en el instante $t_0$), comienza un proceso de **carga o descarga** del condensador interno de la celda.

- En una lectura donde C2 está a $0$, y se conecta con una línea precargada a $1$, el condensador se **descarga** rápidamente. Esto implica una **subida de tensión** en C2, a partir de $t_0$.



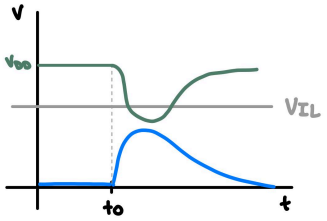

- En una lectura donde C2 está a $1$, y la línea $\overline{BIT}$ se conecta y está a $1$, **no hay transferencia** de carga apreciable, ya que ambos están a la misma tensión. Por tanto, no se observa cambio apreciable de tensión en esa línea. Pero como el nodo C1 está a $0$, entonces ahí sí se produce una **caída de tensión en** $BIT$.

El cambio es rápido pero suave, no es instantáneo porque depende del valor de las resistencias y la capacidad del nodo. Pero ocurre lo bastante rápido como para que el circuito de lectura pueda detectarlo dentro del tiempo de ciclo.

- Cuando se quiere escribir un 1, se pone la línea $BIT$ a $1$ y $\overline{BIT}$ a $0$, y se activa WORD. El nodo que estaba a $0$ (por ejemplo C2) se **carga** rápidamente a $1$,

provocando una **subida de tensión** en C2, mientras que C1 se descarga, estableciendo el nuevo valor estable en la celda.

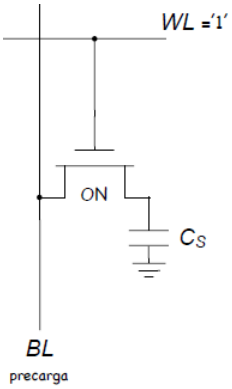
- Cuando se quiere escribir un $0$, se pone la línea $BIT$ a $0$ y $\overline{BIT}$ a $1$, y se activa WORD. El nodo que estaba a $1$ (por ejemplo C2) se **descarga** rápidamente a $0$, provocando una **caída de tensión** en C2, mientras que el nodo opuesto se carga, reforzando el cambio de estado.




## Celda RAM dinámica (DRAM)

Una celda de memoria **RAM dinámica** (DRAM) es un circuito que almacena un solo bit de información durante un tiempo limitado. Se compone únicamente de **dos elementos**:

1. Un **transistor de acceso** (controlado por la línea WL= Word Line).
2. Un **condensador** ($C_S$), que almacena la carga que representa el bit.




Este diseño permite construir celdas más pequeñas y económicas, pero el contenido se pierde con el tiempo y debe refrescarse constantemente.

<u>Modo escritura (guardar un valor)</u>

**Cargar o descargar** el condensador $C_S$ para almacenar un $1$ o un $0$. Se pone el valor deseado (un $1$ lógico = alta tensión, o un $0$ lógico = baja tensión) en la línea de datos: BL (Bit Line). Se activa la línea WL, que abre el transistor de acceso y conecta el condensador $C_S$ con BL. Entonces:

- Si BL está a $1$, $C_S$ se carga completamente.
- Si BL está a $0$, $C_S$ se descarga (se queda sin carga o se pone a tierra).

Después se **desactiva WL**, el transistor se cierra, y el condensador queda aislado, reteniendo su carga durante un cierto tiempo. La información queda

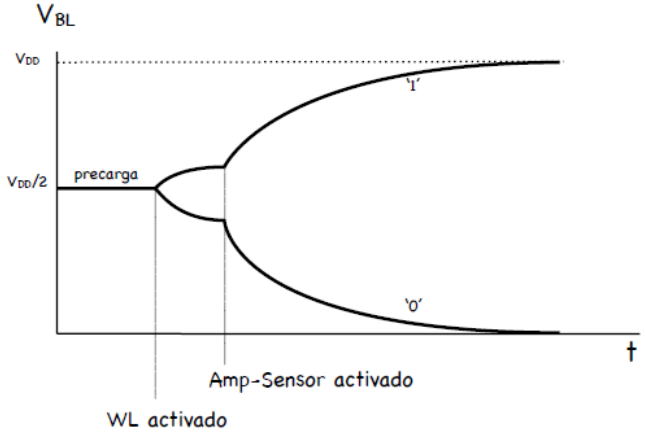
almacenada como una cantidad de carga eléctrica en $C_S$ (se representa un $1$ si es suficiente, o un $0$ si es casi nula).

<u>Modo lectura (consultar el valor)</u>

Detectar cuánta carga queda en $C_S$, sin modificarla demasiado. Se **precarga la línea BL** a un $V_{DD}/2$ (para que cualquier pequeña diferencia causada por $C_S$ se note). Luego, se activa WL, abriendo el transistor. En ese instante, el condensador $C_S$ y la línea BL (que también tiene su propia capacidad, $C_{BL}$, otro condensador) quedan conectados y comparten carga.

- Si $C_S$ estaba **cargado** ($1$), parte de su carga se traspasa a $C_{BL}$, por lo que la tensión de BL **sube** ligeramente.
- Si $C_S$ estaba **descargado** ($0$), la carga de $C_{BL}$ fluye hacia $C_S$, por lo que la tensión de BL **baja** ligeramente.

Esos pequeños cambios (normalmente una diferencia de unos 250 mV) son detectados por un detector sensible a la dirección de cambio de tensión, que compara con un valor de referencia y determina si se leyó un $0$ o un $1$ (de esta forma se restaura el valor lógico de lectura).

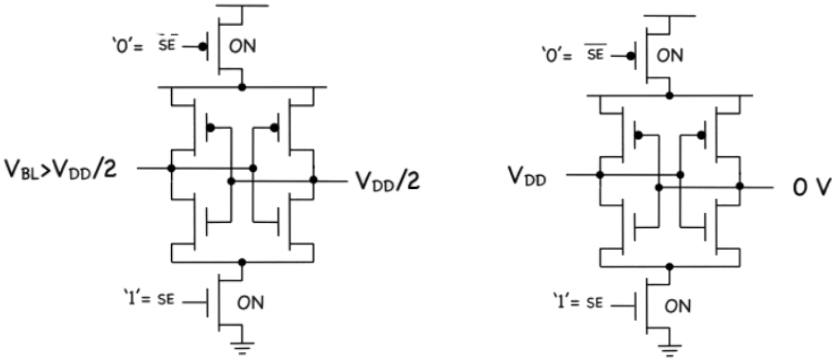



La operación de **lectura es destructiva**, porque al compartir carga entre $C_S$ y $C_{BL}$, el contenido de $C_S$ se modifica, y por tanto se pierde el dato original. Es decir, después de leer, el condensador ya no tiene el mismo nivel de carga que antes. Por eso, cada lectura debe ir seguida de una reescritura del dato, para conservar la información.

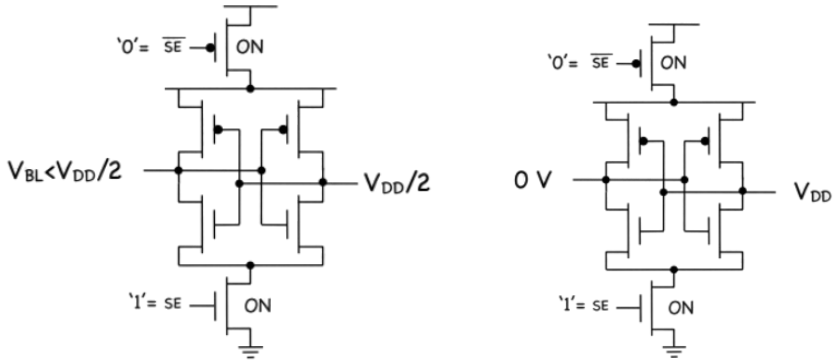
## Circuitos periféricos: Amplificador sensor

Permite **aumentar la velocidad** y **ahorrar área** en las SRAM, además que es **indispensable** para el funcionamiento correcto de las DRAM. Supongamos una implementación para una DRAM:

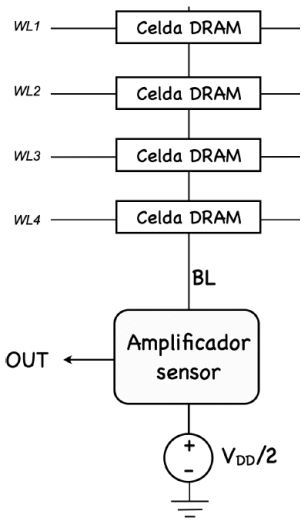
- Si $V_{BL} > \frac{V_{DD}}{2} \Rightarrow V_{OUT} = V_{DD}$.

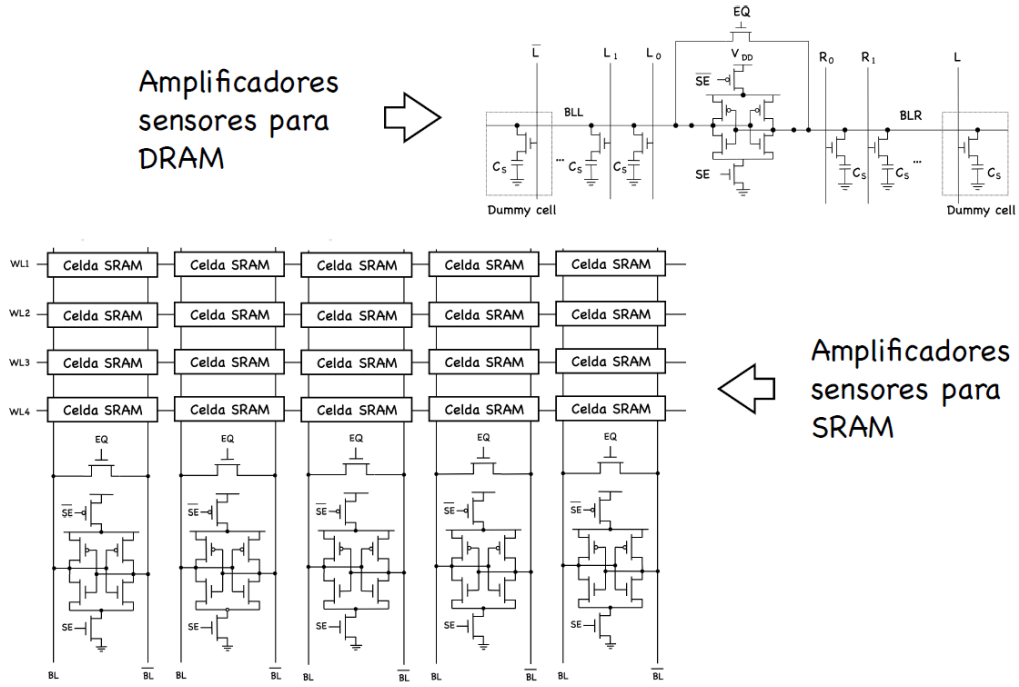



- Si $V_{BL} < \frac{V_{DD}}{2} \Rightarrow V_{OUT} = 0V$.




La implementación de la tensión de referencia es difícil y costosa, pero existe una solución: en lugar de tener un generador de tensión exacta, se puede crear una referencia usando **celdas "fantasma"** (o dummy cells). Estas celdas no almacenan datos reales, pero se comportan como una media entre un $0$ y un $1$, generando así un voltaje de referencia automáticamente durante la lectura.





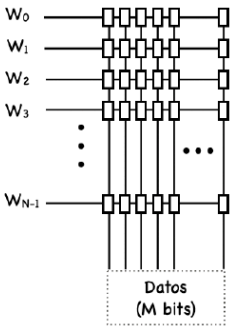

**EQ** es la señal de ecuación, se encarga de precargar BL y $\overline{BL}$ a la misma tensión. Además **SE** y $\overline{SE}$ son señales de selección, que se encargan de activar o desactivar el amplificador (para ahorrar energía).

En los amplificadores sensores para DRAM: Se colocan dos dummy cells en cada columna de memoria, una en un extremo de la línea de bit (BL) y otra en el extremo opuesto de la línea complementaria $\overline{BL}$. Estas dos celdas están diseñadas para **mantener una tensión de referencia** próxima a $V_{DD}/2$. Durante la lectura, mientras una celda real deposita su carga en BL, la dummy cell correspondiente deposita su tensión de referencia en $\overline{BL}$ (o viceversa).

En los amplificadores sensores para SRAM el uso de dummy cells es opcional o innecesario.

## Circuitos periféricos: decodificadores

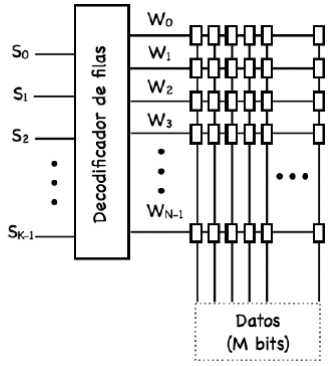
Los **decodificadores** son circuitos que reciben una dirección (en binario) y activan sólo una de muchas líneas posibles.




Es una **arquitectura intuitiva** de una memoria NxM, es decir:

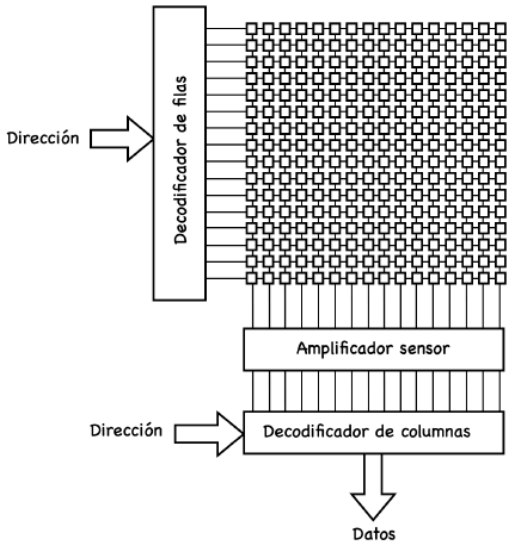
- Tiene N palabras (filas), o sea, N posiciones donde guardar datos.
- Cada palabra contiene M bits (las columnas).

Sin decodificador y se necesitara activar cada fila por separado, se necesitarían **N líneas de selección**.




Usando un decodificador se reduce el número de líneas de selección. N palabras necesitan K líneas de selección ($K = log_2 N$).

En los chips de memoria (como DRAM ou SRAM), os datos están organizados en forma de matriz: Las **filas** (horizontal), representa cada una a unha palabra de datos, y las **columnas** (vertical), cada una representa un bit dentro da palabra.



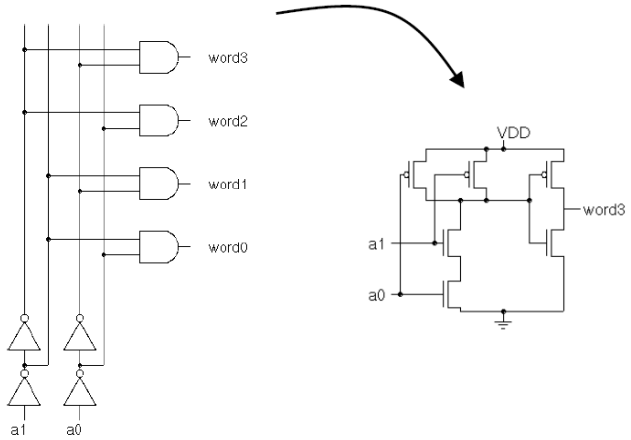

En una situación óptima el **número de filas es igual al número de columnas** (la dimensión horizontal = dimensión vertical), de modo que el "layout" sea más regular y tenga una mayor velocidad.

Por lo que se necesitan **dos decodificadores**, uno por filas y otro por columnas. Dada una dirección de memoria, el decodificador de filas selecciona la línea de memoria que se requiere, que pasa al amplificador sensor (para que devuelva valores lógicos correctos) y el decodificador de columnas, dada la misma dirección, seleccione los datos correspondientes.

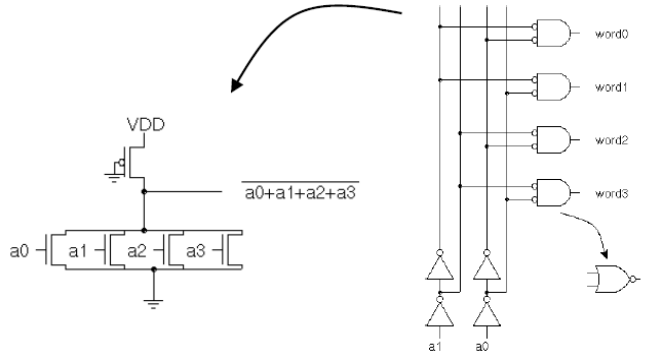
## — Decodificadores de filas.

Se usan dos tipos de puertas para este tipo de decodificadores:

1. **Puertas NAND estáticas**: Son puertas NAND hechas con tecnología CMOS complementario. Tiene mucha **limitación práctica**: si tienen **muchas entradas** (más de 5 o 6), se vuelven lentas y grandes, porque cada entrada añade más transistores en serie (en el camino de conducción), lo que aumenta la resistencia y retrasa la salida.



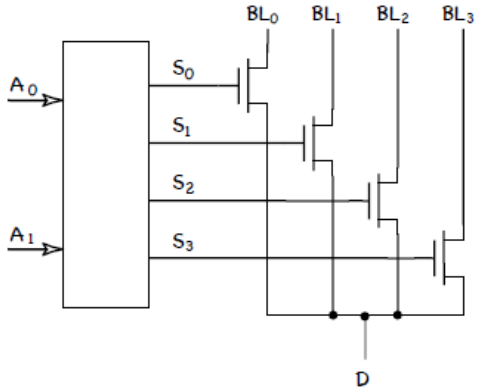
2. Puertas **NOR pseudo-NMOS**: Son puertas NOR especiales, en las que la parte PMOS (la del transistor de arriba) es reemplazada por un **solo transistor activo siempre** (como una resistencia controlada). Son más rápidas que las NAND estáticas cuando se usan con muchas entradas. Además, ocupan menos espacio en el chip.



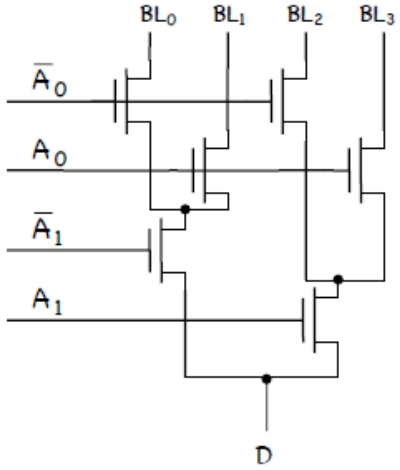

**— Decodificadores de columnas.**

Existen distintas formas de implementar un decodificador de columnas, y cada una tiene ventajas e inconvenientes.

1. Con **transistores de paso directamente en las líneas de bit**: Tiene mayor velocidad y sólo se incluye un transistor extra en la vía de datos crítica. Pero el número de transistores es elevado.

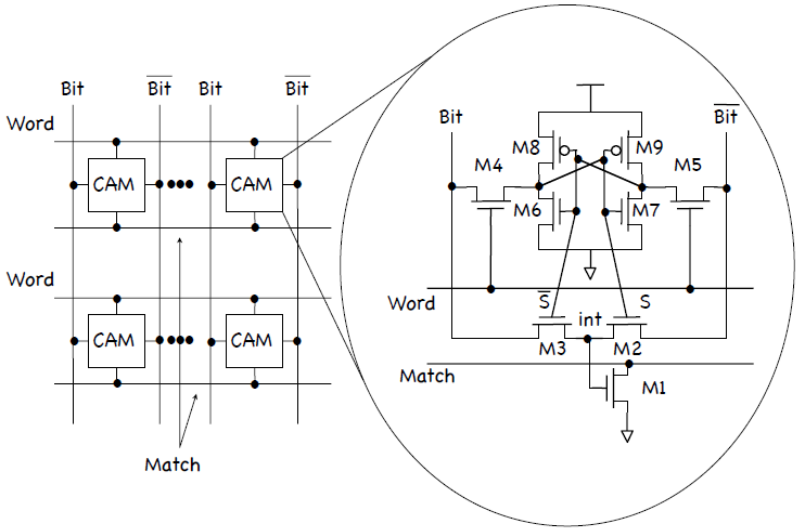



2. Con **lógica combinacional**: Número reducido de transistores, pero el retardo es elevado y prohibitivo para grandes decodificadores.



## Memorias CAM (Content Addressable Memory)

Una **memoria CAM** (Content Addressable Memory, o memoria de acceso por contenido) es un tipo de memoria que permite buscar datos en base al **contenido** de las celdas, no en función de una dirección fija.



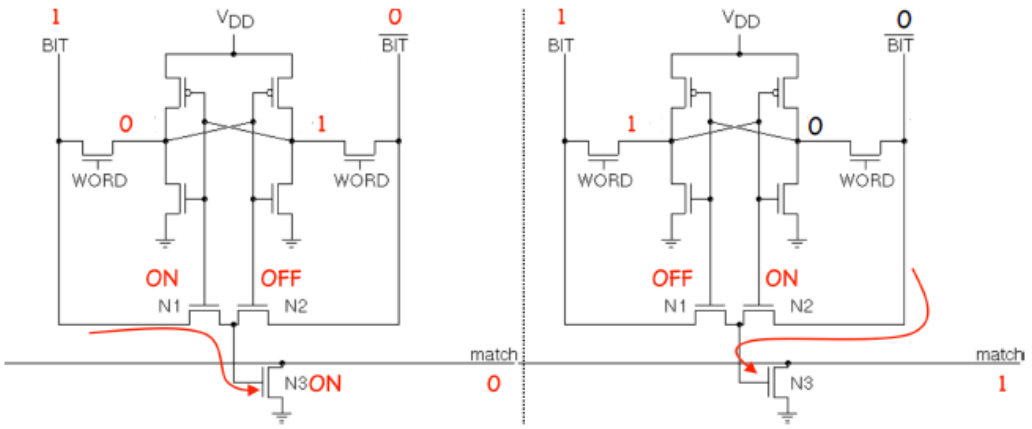

El proceso de búsqueda en una memoria CAM es diferente al de una memoria tradicional.

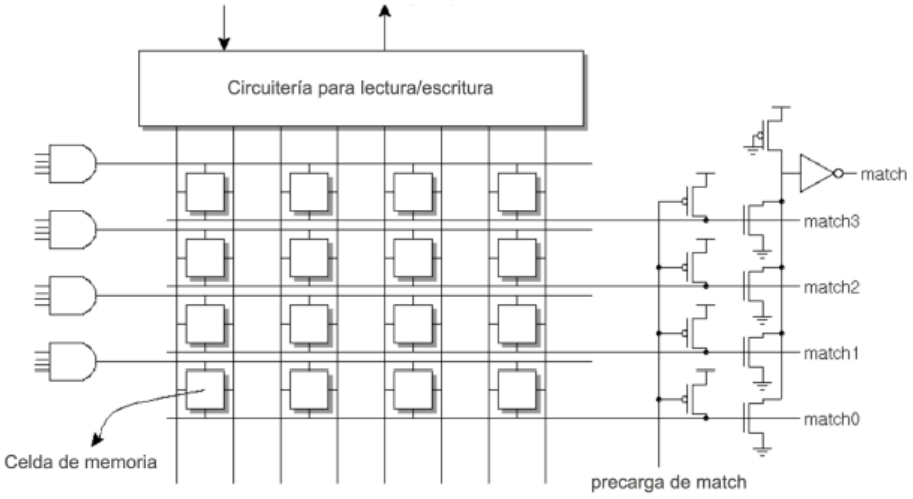
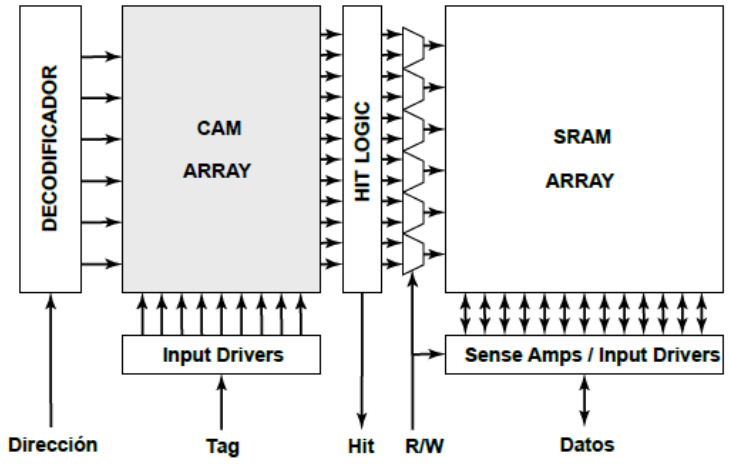
1. Se **precarga siempre el MATCH** a $1$.

2. Buscar en **líneas BIT**:
   a. La **operación MATCH** empieza cuando le envías un **dato** que deseas buscar dentro de las celdas de la memoria CAM. Este dato es comparado con los valores almacenados en cada celda de la memoria.
   b. En términos prácticos, cada celda de la memoria tiene un valor ($0$ o $1$) y el dato a buscar se envía para compararlo con los valores almacenados en las celdas.

3. **Comparación** del dato con las celdas:
   a. La memoria CAM compara el dato buscado con el contenido de cada celda. Cada celda tiene una lógica para realizar esta comparación, y si el contenido de una celda **coincide** con el dato que buscas, esa celda **se marcará como un "match"** (coincidencia).

4. **Estado** de la celda (WORD = $0$):
   a. Si el **dato no está en la celda**, se activa una señal, **N3 = $1$ (ON)** y Match se pone a $0$, que indica que no ha habido coincidencia.
   b. Si el dato está en la celda, la señal **Match** se pone a $1$ (indica que se ha encontrado una coincidencia).




Búsqueda 1; Valor almacenado 0 Búsqueda 1; Valor almacenado 1

En una memoria CAM, todas las celdas de una palabra están **conectadas a una línea MATCH común**, que se gestiona mediante una puerta NOR distribuida (dinámica o pseudo-NMOS). Las señales de coincidencia generadas por las líneas match(i) determinan qué palabra (WORD) de la memoria RAM se activa, indicando la ubicación del dato buscado.